מבוא לקריפטוגרפיה - הרצאה 5

7 בנובמבר 2026

גיא יער־און

סיכום זה נכתב במהלך סמס א' תשפ"ז (2027) במהלך הרצאותיו של ד"ר גלעד אשרוב. הסיכום בין היתר מבוסס על סיכום הרצאות ותרגולים של ד"ר גלעד אשרוב משנת 2024.
השימוש בסיכום באחריותכם בלבד.

תוכן עניינים

0.1 בטיחות כנגד התקפות צופן נבחר Ciphertext Chosen Attacks 1
0.2 סכימות לאימות הודעות 2
0.2.1 בניית סכימה לאימות הודעות 5
0.2.2 הרחבת דומיין לMac 7
0.2.3 CBC MAC 7
0.2.4 תקשורת פרטית ומאומתת - לשים לב אם בהרצאה הוא מפרמל יותר! 8
0.3 תרגילים 9

0.1 בטיחות כנגד התקפות צופן נבחר Ciphertext Chosen Attacks

התקפת הPadding Oracle עליה דיברנו בהרצאה הקודמת הכינה אותנו לדון בהתקפות נגד צופן נבחר - CCA. למה צריך הגדרה חדשה? ובכן, עד כה ראינו שלושה סוגי יריבים מהחלש לחזק:

1. מאזין (eav): הוא רק רואה צפנים שעוברים בערוץ.
2. טקסט נבחר (CPA): בנוסף, הוא יכול לבקש הצפנות של הודעות לבחירתו באמצעות גישות אורקל הצפנה.
3. צופן נבחר (CCA): בנוסף לכל יכולת שאר היריבים, הוא יכול לבקש פענוחים של צפנים לבחירתו - אורקל פענוח.

התקפת הPadding oracle הראתה מדוע צריך את הגדרת היריב השלישי: CBC למשל בטוח CPA ובכל זאת נשבר, כי התוקף קיבל מידע על פענוח של צפנים שהוא בנה. CPA לא מכסה זאת, ולכן יש צורך בהגדרה חדשה. נגדיר כעת את הניסוי שדומה לניסויים קודמים עם שינוי אחד בודד - הוספת אורקל פענוח.

הניסוי $\mathrm{PrivK}^{\mathrm{cca}}_{\mathcal{A},\Pi}(n)$:
1. מפתח k מיוצר ע"י הרצת $\mathrm{Gen}(1^n)$.
2. היריב $\mathcal{A}$ מקבל כקלט 1^n וגישת אורקל ל:
   א. אורקל הצפנה - $\mathrm{Enc}_k(\cdot)$, עם כל שאילתא m היריב מקבל צופן $c \leftarrow \mathrm{Enc}_k(m)$.
   ב. אורקל פענוח - $\mathrm{Dec}_k(\cdot)$. עם כל שאילתא c היריב מקבל את התוצאה של $\mathrm{Dec}_k(c)$.
3. היריב מוציא זוג הודעות m_0, m_1 מאותו אורך.
4. ביט $b \in \{0,1\}$ נבחר באקראי וצופן אתגר $c \leftarrow \mathrm{Enc}_k(m_b)$ מחושב ונשלח ל$\mathcal{A}$.
5. היריב $\mathcal{A}$ ממשיך לקבל גישות אורקל ל$\mathrm{Enc}_k(\cdot), \mathrm{Dec}_k(\cdot)$ תחת האילוץ שאסור לו לשאול על c.
6. לבסוף, $\mathcal{A}$ מוציא ביט b'.
7. פלט הניסוי מוגדר להיות 1 אם ורק אם $b = b'$.

נשים לב כי המגבלה בשלב 5 טריוואלית - אילולא היינו אוסרים עליו לשאול על c, הוא היה שולח את c לאורקל הפענוח ומקבל חזרה את m_b. משווה לזוג ההודעות ויודע אם להחזיר 1 או 0. אבל, מותר לו לשאול על כל צופן אחר: כולל כאלו הדומים מאוד לc.

הגדרה 1. סכימת הצפנה סימטרית $\Pi = (\mathrm{Gen}, \mathrm{Enc}, \mathrm{Dec})$ משיגה בטיחות כנגד התקפת צופן נבחר אם לכל יריב PPT, $\mathcal{A}$ קיימת פונקציה זניחה negl כך ש:

$$\Pr\left[\mathrm{PrivK}^{\mathrm{cca}}_{\mathcal{A},\Pi}(n) = 1\right] \leq \frac{1}{2} + \mathrm{negl}(n)$$

למעשה, הגדרה זהה לגמרי להגדרות הקודמות שראינו חוץ מהניסוי שמשתנה.
ובכן, למה אנחנו מגדירים את ההגדרה הזו? נכון שבעולם האמיתי - אין אורקל פענוח "ידידותי". אבל, יש מצבים שמאוד מתקרבים לאורקל שכזה. לכן המודל לוקח את המקרה הגרוע ביותר עבורינו, כאשר היריב יכול לפענח הכל חוץ מc כמובן על מנת להיות בטוח. נציג שתי דוגמאות:
1. תקשורת עם בנק: יריב משתלט על הערוץ ושולח צפנים משלו לבנק בשמך. הבנק מפענח אותם, ובדרך כלל מחזיר הודעת שגיאה או אם במקרה הצופן התפענח למשהו תקין מתנהג אחרת. כל תגובה שכזו, מדליפה מידע על הפענוח בדיוק כמו בPaddind Oracle שתגובת הבנק עזרה להבין את ההודעה. הבנק משמש בלי כוונה כאורקל פענוח חלקי.
2. הצפנה כאימות: נניח כי רוצים לבדוק שמישהו מחזיק את המפתח k: שולחים לו הודעה אקראית מוצפנת r, ומצפים שיחזיר r. אבל אז, הצד השני בעצם מפענח כל צופן ששלחו לו ומחזיר את התוצאה. תוקף יכול לנצל זאת - ישלח צופן c שהוא מעוניין לפענח, ולקבל חזרה את $\mathrm{Dec}_k(c)$.

כל הסכימות שראינו עד שלב זה בקורס, נשברות עם CCA. מדוע? נתבונן בסכימה $(r, \mathrm{F}_k(r) \oplus m)$ כדוגמה. היריב מנצל תכונה מסוכנת מאוד - הוא יכול לשנות את הצופן בצורה צפויה. מה הכוונה? היריב בוחר m_0, m_1 ומקבל צופן אתגר $c = (c_1, c_2)$ אשר $c_1 = r$ ו$c_2 = F_k(r) \oplus m_b$
כעת, הוא יבנה צופן חדש, שונה מc:

$$c' = (c_1, c_2 \oplus 1^n)$$

כלומר, הוא הופך את כל הביטים של החלק השני. כיוון שבהכרח $c \neq c'$: מותר לו לשאול את אורקל הפענוח אודות c'. מה יחזור?

$$\mathrm{Dec}_k(c') = F_k(c_1) \oplus (c_2 \oplus 1^n) = (F_k(c_1) \oplus c_2) \oplus 1^n = m_b \oplus 1^n$$

הסבר: בהינתן זוג (a, b) אזי $\mathrm{Dec}_k((a, b)) = F_k(a) \oplus b$. מדוע זה נכון? הוא מחזיק את k לכן יכול אכן לחשב עם הפונקציה הפסאודו־רנדומית, והרי מתקיים $(m \oplus F_k(r)) \oplus F_k(r) = m$ אכן. נשים לב כי תמיד $a = r$.
כעת, היריב יבצע Xor עם 1^n על מנת לקבל את m_b, ורואה מיד אם זה m_0 או m_1 ומחזיר תשובה בהתאם: ניצחון בהסתברות 1.
כלומר, הרעיון המרכזי הוא שהיריב לא שבר את ההצפנה ישירות - הוא לקח את צופן האתגר, שינה אותו קצת בצורה שהוא שולט בה, שאל על הגרסה החדשה שמותרת וחילץ את ההודעה. כל הסכימות שראינו גמישות ככה - ולכן נשברות.

לכן נשאל את עצמנו: כיצד בונים סכמות בטוחות CCA? האינטואיציה למעשה, היא בדיוק לחסום את מה שהיריב מנסה לעשות - שהיריב לא יוכל ליצור צפנים חוקיים בעצמו. אם כל שינוי שהוא עושה בצופן הופך אותו ללא חוקי, אורקל הפענוח פשוט יחזיר שגיאה על כל מה שהוא ממציא ולא יתן לו שום מידע. מה שצריך בשביל זה - נקרא אימות: מנגנון שמוודא שרק מי שמחזיק את המפתח יוכל ליצור צופן תקין, זה בדיוק מה שמיד נעסוק בו: וממנו בונים הצפנה בטוחה CCA.

0.2 סכימות לאימות הודעות

עד כה דיברנו על סודיות - שאף אחד לא יקרא את ההודעה. כעת עוברים לבעיה אחרת לגמרי, אימות: לוודא שאף אחד לא שינה את ההודעה ושהיא באמת הגיעה ממי שהיא טוענת. כלומר ישנן 2 בעיות נפרדות: סודיות ואימות הן 2 מטרות שונות, ואחת מהן לא גוררת את השנייה.
סודיות: היריב לא יכול לקרוא את ההודעה. זה מה שהצפנה נותנת.
אימות: היריב לא יכול לזייף או לשנות הודעה מבלי שיתפס.
אפשר שתהיה סודיות ללא אימות, ולהפך. מדוע הצפנה לבדה לא מאמתת? נניח כי בוב שולח לבנק הודעה מוצפנת - "העבר 1000$ לאליס". אליס יושבת על הערוץ, היא לא יכולה לקרוא את ההודעה כי היא מוצפנת ויש סודיות - ובכל זאת היא יכולה לשנות

אותה. ניקח את הסכמה $(r, F_k(r) \oplus m)$. אליס רואה את החלק השני - $F_k(r) \oplus m$: היא לא יודעת מהו m. אבל אם היא עושה XOR של משהו לחלק הזה היא משנה את m באותו אופן בדיוק.

$$(F_k(r) \oplus m) \oplus \Delta = F_k(r) \oplus (m \oplus \Delta)$$

כלומר ההודעה שתפוענח אצל הבנק היא $(m \oplus \Delta)$ במקום m. אם אליס יודעת גם את מבנה ההודעה - איפה נמצא הסכום, היא יכולה להפוך את $1000 ל$100000, בלי לדעת כלום על התוכן. זוהי אותה הגמישות שראינו בהתקפת CCA ובהתקפת Padding Oracle: הצפנה מגנה על מה שההודעה אומרת, אך לא מונעת מהיריב לחבל בה. מה אנחנו רוצים? מנגנון שבו רק מי שמחזיק את המפתח k יוכל לייצר הודעה תקינה. אם היריב ישנה ולו ביט אחד, או ינסה לזייף הודעה הבדיקה תכשל והבנק ידחה. זה נקרא MAC - Message Authentication Code ועל זה נדון עד סוף ההרצאה הנוכחית. הוא גם הכלי ממנו בונים הצפנת CCA בטוחה: כמו שאמרנו קודם - אם היריב לא יוכל בכלל ליצור צפנים תקינים, אורקל הפענוח לא יעזור לו.

נעיר כי יש תחום אחר ושמו "קודים לתיקון שגיאות" שגם עוסק בהודעות שהשתנו - אך המטרה הפוכה. בתיקון שגיאות מניחים שרעש אקראי שיבש את ההודעה ורוצים לשחזר את המקור - זה למשל מה שראינו (התותחים ביותר שלקחו "מבנים אלגבריים") בקורס. ההנחה בתיקון שגיאות שאין יריב אלא טבע בלבד. באימות (MAC) מניחים שיש יריב זדוני ששינה את ההודעה בכוונה ולא מנסים לתקן, אלא רק לזהות שמשהו שונה ולדחות.

מכאן, נעבור להגדרות הפורמליות.

הגדרה 2. קוד לאימות הודעות (Code Authentication Message) מורכב משלשה של אלגוריתמים $(\mathrm{Gen}, \mathrm{Mac}, \mathrm{Vrfy})$:

1. אלגוריתם ליצירת מפתח Gen לוקח כקלט פרמטר בטיחות 1^n ומוציא מפתח k כך ש$|k| \geq n$.
2. אלגוריתם ליצירת תג Mac מקבל כקלט מפתח k והודעה $m \in \{0,1\}^*$ ומחזיר תג t. מכיוון שהאלגוריתם יכול להיות רנודמי, נכתוב $t \leftarrow \mathrm{Mac}_k(m)$
3. אלגוריתם דטרמיניסטי Vrfy מקבל כקלט מפתח k, הודעה m ותג t ומחזיר האם התג t הוא תיקני ביחס להודעה m והמפתח k - תשובה בינארית. נכתוב $b = \mathrm{Vrfy}_k(m,t)$.

נדרוש כי לכל n, לכל k שהוא פלט של $\mathrm{Gen}(1^n)$ ולכל הודעה $m \in \{0,1\}^*$ מתקיים:

$$\mathrm{Vrfy}_k(m, \mathrm{Mac}_k(m)) = 1$$

כלומר, בדיוק כמו שהצפנה היא שלשת אלגוריתמים כך גם אימות, עם דרישה הגיונית בסיום בדומה להצפנה. התג הוא כמו חותמת שעווה על מכתב, Mac היא החותמת שרק בעלת המפתח יכול להטביע. וVrfy הוא הבדיקה שהחתימה אמיתית. כיצד משתמשים בזה? בוב שולח לאליס את ההודעה יחד עם התג (m,t). אליס, אשר מחזיקה את אותו k מריצה $\mathrm{Vrfy}_k(m,t)$: אם זה מחזיר 1 היא יודעת שההודעה אמיתית ולא שונתה. נשים לב כי m נשלחת גלויה - Mac לא מצפין **כלום**, אלא רק מאמת.

הגדרה 3. וידוא קנוני: בסכימות שבהן Mac הוא דטרמיניסטי ולא רנדומי, כלומר אותה הודעה נותנת תמיד אותו תג, אזי אין צורך להגדיר את Vrfy בנפרד כי יש דרך סטנדרטית לעשות זאת - Vrfy פשוט מחשב את התג בעצמו ומשווה. כלומר:

$$\mathrm{Vrfy}_k(m,t) = \text{Compute } t' = \mathrm{Mac}_k(m) \text{ and return 1 iff } t' = t$$

רק אם Mac דטרמיניסטי אזי ניתן לחשב ולהשוות כך. עם הסתברותי זה לא יעבוד כי יתכן שהתג שמחושב יהיה שונה מזה שהתקבל, צריך שיטה שונה.

כעת השאלה המרכזית - מתי Mac בטוח? האינטואיציה: מבלי לדעת את k, היריב לא יכול לזייף תג תקין על שום הודעה חדשה גם אם ראה כבר המון זוגות תקינים (m,t) ואפילו של הודעות שהוא בחר. כלומר דורשים הרבה - נותנים ליריב כל מה שהוא רוצה, אפשרות לקבל תגים להודעות לבחירתו וכו' והוא עדיין לא מצליח לזייף תג אפילו על הודעה אחת חדשה. נגדיר את הניסוי באופן הבא:

הניסוי $\text{Mac-forge}_{\mathcal{A},\Pi}(n)$:
1. המפתח k מיוצר ע"י $\text{Gen}(1^n)$.
2. היריב $\mathcal{A}$ מקבל קלט 1^n וגישת אורקל ל$\text{Mac}_k(\cdot)$, היריב מתישהו מוציא זוג (m,t). נסמן בQ את אוסף כל השאילתות ש$\mathcal{A}$ שואל את האורקל שלו.
3. נאמר כי $\mathcal{A}$ מצליח אם ורק אם:
א. $\text{Vrfy}_k(m,t)=1$ - כלומר היריב מוציא תג חוקי.
ב. $m \notin Q$. כלומר, התג הנ"ל הוא על הודעה שלא נשאלה אף פעם.
במקרה שכזה פלט הניסוי הוא 1, ואחרת אפס.

מדוע הניסוי מוגדר כך, כלומר מדוע אלו שני התנאים להצלחת היריב? תנאי א' יותר ברור - אם התג לא תקין, הוא לא זייף כלום. תנאי ב' הוא העיקר: בלעדיו הזיוף היה טריוואלי - היריב שואל את האורקל על הודעה כלשהי m, מקבל תג תקין t ומחזיר את (m,t). אבל אין בזה שום זיוף - הזיוף האמיתי הוא ליצור תג להודעה שמעולם לא ראה. זה מה שדורש לדעת את k. העקרון הוא שנגדיר בטיחות אם היריב לא הצליח לגלות אף זוג של הודעה ותג (m,t).

הגדרה 4. סכימה לאימות הודעות $\Pi=(\text{Gen},\text{Mac},\text{Vrfy})$ היא בטוחה כנגד זיוף אם לכל יריב PPT, $\mathcal{A}$ קיימת פונקציה זניחה negl כך ש:

$$\Pr\left[\text{Mac-forge}_{\mathcal{A},\Pi}(n)=1\right]\leq \text{negl}(n)$$

נשים לב, כאן אין עוד $\frac{1}{2}+\text{negl}(n)$, כי אין מה לנחש. זיוף מוצלח הוא אירוע שלא אמור לקרות כמעט לעולם.

התקפת Replay Mac לא מגן מפני שליחה חוזרת של הודעה ישנה - זו מגבלה חשובה שיש להכיר. מה הכוונה? אליס שולחת לבנק "העבר 100$ לבוב", עם תג תקין. בוב רואה את ההודעה והתג, והוא פשוט שולח את אותו הזוג (m,t) שוב לבנק. תג תקין לגמרי, והבנק יעביר כתוצאה מכך 200$ - פעמיים קיבל את ההודעה.
למה Mac לא תופס את זה? כי בוב לא זייף כלום - הוא לא יצר תג חדש, רק העתיק הודעה תקינה ושלח אותה שוב עם תג תקין. ההגדרה מבטיחה שאי אפשר לזייף תג על הודעה חדשה, אך לא אומרת כלום על הודעה קיימת. כיצד מתגוננים מכך? זה לא תפקיד של Mac אלא של האפליקציה: מוסיפים להודעה למשל מס' סידורי או חותמת זמן, ואז למרות שנשלחה לכאורה אותה הודעה מאחורי הקלעים היא מתורגמת שונה - עם תג שונה, ולכן הבנק ידחה הודעה עם מס' סידורי שכבר ראה. כך פתרנו את הבעיה ואותה הודעה לא תתקבל פעמיים.

סכמות לאימות הודעות חזקות (Strong Mac) ההגדרה הרגילה, אוסרת לזייף תג על הודעה חדשה. אך אם Mac הוא אלגוריתם הסתברותי, לאותה הודעה ישנם הרבה תגים תקינים שונים. אז אפשר לדרוש יותר:
Mac רגיל: היריב לא יכול לייצר תג תקין על הודעה שלא ראה.
Strong Mac: היריב לא יכול לייצר זוג חדש (m,t) תקין, גם אם m עצמה כבר נראתה: כל עוד התג t הנ"ל חדש.
כלומר פורמלית, ההבדל הוא רק מה שומרים בQ: בMac הרגיל שומרים הודעות ובחזק יותר: שומרים זוגות (m,t). היריב מנצח אם הוציא זוג תקין שאינו בQ.
אם Mac הוא דטרמיניסטי אז אין הבדל בין השניים, כי אז לכל הודעה יש בדיוק תג אחד.
לסיכום: הכל תלוי במה היריב צריך בשביל לנצח. ברגיל, היריב מנצח אם הוא מייצר תג תקין עבור הודעה חדשה. כזו שלא שאל עליה עוד. בחזק, הוא מנצח אם יצר זוג (m,t) חדש לגמרי - כלומר הוא יכול לנצח גם על הודעות שכבר ראה, בתנאי שאת התג t הוא לא ראה. כלומר גם מציאת תג נוסף לאותה הודעה נחשב זיוף.

התקפת זמן (timing attack) נזכר בוידוא הקנוני - Vrfy מחשב $t'=\text{Mac}_k(m)$ ומשווה לt שקיבל. איך משווים זוג מחרוזות בקוד רגיל? עוברים תו־תו, וברגע שמוצאים תו שאיננו מתאים עוצרים ומחזירים לא. וזו בדיוק הבעיה: זמן. הזמן שלוקח לוידוא להחזיר לא מגלה עד איזה תו ההשוואה הצליחה. אם התו הראשון שגוי - התשובה מיידית. אם 5 התווים הראשונים נכונים והשישי שגוי, זה יקח קצת יותר זמן. כיצד היריב ינצל את זה? הוא מגלה את התג הנכון תו אחר תו.
- נניח כי הוא יודע כבר את i התווים הראשונים. כדי לגלות את התו ה$i+1$, הוא מכסה את כל האפשרויות לתו הזה.
- לכל ניחוש שגוי, ההשוואה נעצרת בתו ה$i+1$. לניחוש הנכון, ההשוואה ממשיכה לתו ה$i+2$ ולכן לוקחת קצת יותר זמן (!)
- היריב מודד את כל הזמנים, מזהה את הניחוש האיטי יותר וזה התו הנכון - וכך ממשיכים הלאה.

היתרון העצום: במקום לנחש את כל התג בבת אחת שזה $(A)^n$כאשר A הוא מס' האפשרויות לכל תו, הוא מגלה תו אחר תו : A נסיונות לכל אות ולכן זמן הריצה שלו הוא $\mathcal{O}(A\cdot n)$. ההבדל הוא בין אקספוננציאלי ללינארי - זה משמעותי מאוד.

כיצד אפשר להגן? לוודא שהשוואה תקח אותו זמן בדיוק, לא משנה כמה תווים מתאימים. בפרט, לא לעצור מוקדם כשמוצאים אי התאמה אלא תמיד לעבור על כל התווים: אופטימיזציה תמימה כמו יציאה מלולאה שמלמדים סטודנטים בשנה א' שכדאי לעשות ־ בקריפטוגרפיה יכולה לפתוח פרצה קריטית.

0.2.1 בניית סכימה לאימות הודעות

עבור גודל בלוק אחד, הבניה של סכימה היא פשוטה: מפעילים פונקציה פסאודו־אקראית. נגדיר:

הגדרה 5. תהי F פונקציה פסאודו־אקראית. נגדיר Mac עבור בלוקים בגודל קבוע מאורך n בצורה הבאה:
1. $\mathrm{Gen}\left(1^n\right)$: מחזיר מפתח $k \in \{0,1\}^n$
2. $\mathrm{Mac}\left(k,m\right)$: כאשר $k,m \in \{0,1\}^n$, נחזיר $t = F_k\left(m\right)$.
3. $\mathrm{Vrfy}_k\left(m,t\right)$: נחשב $t' = F_k\left(m\right)$ ונחזיר 1 אם ורק אם $t = t'$.

הגדרה 6. תחת ההנחה כי F היא פונקציה פסאודו־אקראית, אזי הבניה לעיל היא סכימה לאימות הודעות עבור הודעות מאורך n.

הוכחה:

ראשית ־ מדוע שהבנייה לעיל תעבוד בכלל? F_k נראית אקראית לגמרי למי שלא יודע את k, אז בשביל היריב $F_k\left(m\right)$ הוא ערך אקראי לגמרי ואין לו דרך לנחש אותו, כי כדי לזייף תג על הודעה היריב צריך לנחש את המחרוזת: וזה קורה בהסתברות 2^{-n}. נשים לב שזו הבניה הטבעית ביותר שניתן לדמיין, אין r אקראי ואין Xor כי לא מצפינים כלום, רק רוצים ערך שבעל המפתח בלבד יכול לחשב. ההוכחה עובדת בשני שלבים לפי המתכונת:

1. העולם האידיאלי הוא בטוח: נוכיח כי אם הסכימה הייתה משתמשת בפונקציה אקראית לחלוטין ולא פסאודו־אקראית, אזי הסכימה בטוחה.
2. לא ניתן להבחין בין העולם האמיתי לאידיאלי: נוכיח שאין שום יריב שמסוגל להבחין אם הסכימה שהוא משתמש בה משתמשת בפונקציה פסאודו־אקראית או אקראית. אם הוא יכול להבחין בין זוג הסכמות: הוא יכול להבדיל בין פונקציה אקראית לפונקציה פסאודו אקראית.

חלק ראשון:

תהי $\tilde{\Pi} = \left(\tilde{\mathrm{Gen}}, \tilde{\mathrm{Mac}}, \tilde{\mathrm{Vrfy}}\right)$ סכימה לא יעילה שמשתמשת בפונקציה אקראית לחלוטין f במקום בפונקציה פסאודו־אקראית. כלומר ב$\tilde{Gen}$ תבחר פונקציה אקראית $f \in \mathrm{Func}_n$. כעת, $\tilde{\mathrm{Mac}_k}\left(m\right) = f\left(m\right)$ שהרי מוציא את פלט הפונקציה, ו$\tilde{\mathrm{Vrfy}}$ היא פונקציית וידוא קנונית ־ שהרי Mac הוא כעת כמו דטרמיניסטי, שכן פונקציה אקראית לחלוטין היא בפועל פונקציה דטרמיניסטית.

נניח כי היריב מוציא זוג $(m,t) \notin Q$ במשחק $\text{Mac-forge}_{\mathcal{A},\tilde{\Pi}}\left(n\right)$. הרי לא שאלנו עוד על m, ולכן בהכרח t עדיין לא נבחר. בשביל שהיריב ינצח צריך להתקיים כי $f\left(m\right) = t$ כאשר t הוא ערך קבוע. הסיכוי כי f שהיא פונקציה אקראית לגמרי תמפה את m לt הוא 2^{-n}. לכן:

$$\Pr\left[\text{Mac-forge}_{\mathcal{A},\tilde{\Pi}}\left(n\right) = 1\right] = \Pr\left[f\left(m\right) = t\right] = 2^{-n}$$

בחלק השני של ההוכחה, נוכיח כי כל יריב לא מסוגל להבחין אם הוא בניסוי $\text{Mac-forge}_{\mathcal{A},\tilde{\Pi}}\left(n\right)$ או בניסוי $\text{Mac-forge}_{\mathcal{A},\Pi}\left(n\right)$. כלומר נוכיח כי לכל יריב $\mathcal{A}$ שהוא PPT קיימת פונקציה זניחה negl כך ש:

$$\left|\Pr\left[\text{Mac-forge}_{\mathcal{A},\tilde{\Pi}}\left(n\right) = 1\right] - \Pr\left[\text{Mac-forge}_{\mathcal{A},\Pi}\left(n\right) = 1\right]\right| \leq \mathrm{negl}\left(n\right)$$

בשלילה שלא, אזי קיים יריב $\mathcal{A}$ שיודע להבדיל בין העולמות. על מנת לעשות זאת, נקבע יריב כלשהו $\mathcal{A}$ ונבנה ממנו מבחין D לפונקציה הפסאודו אקראית ־ נראה שלמעשה באמצעותו ניתן להבדיל בין פונקציה פסאודו אקראית לאקראית.

קלט: המבחין יקבל כקלט 1^n.

גישת אורקל: המבחין מקבל גישת אורקל $\mathcal{O}\left(\cdot\right)$ או לפונקציה פסאודו אקראית $F_k\left(\cdot\right)$ או לפונקציה אקראית $f\left(\cdot\right)$. מטרתו של המבחין היא להבין באיזה מהמצבים הוא נמצא.

המבחין:

1. נסמלץ ל$\mathcal{A}$ את המשחק $\mathrm{Mac}-\mathrm{forge}$.

נריץ את אלגוריתם היריב $\mathcal{A}\left(1^n\right)$ ונסמלץ לו גישת אורקל ל$\mathrm{Mac}_k\left(\cdot\right)$ או $\tilde{\mathrm{Mac}}\left(\cdot\right)$ באופן הבא: כאשר $\mathcal{A}$ שואל על הודעה m, והמבחין מחזיר לו את התג $t = \mathcal{O}\left(m\right)$ (דהיינו, תשובת האורקל לשאלה m) אזי המבחין מוסיף כל שאילתא m לקבוצה Q שבהתחלה ריקה.

2. בשלב מסוים היריב $\mathcal{A}$ יחזיר זוג (m,t). המבחין יבדוק כי:
   א. $m \notin Q$

ב. $t = \mathcal{O}(m)$
אם כן, הוא יחזיר 1 - פסאודו אקראי, ואחרת יחזיר 0: אקראית.
נרצה לפני הניתוח להסביר מדוע בחרנו להתבונן במבחין שכזה בשיטת ההוכחה הזו. מה אנחנו באמת רוצים? להוכיח כי העולם האמיתי והאידיאלי דומים, כלומר שהיריב $\mathcal{A}$ מזייף כמעט באותה הצלחה בשניהם. הבעיה: כיצד מוכיחים "דומים"? אין לנו דרך ישירה - לכן משתמשים בטריק, מוכיחים כי אם הם היו שונים, היה קורה משהו בלתי אפשרי. מה ההבדל היחיד בין העולמות? הפונקציה - F_k לעומת f. אז אם נצליח להפוך את השאלה "$\mathcal{A}$ מתנהג אחרת" ל"מבחין שמבדיל בין F_k לf" הגענו לסתירה. לכן בונים את D המבחין בצורה זו: הוא לוקח את $\mathcal{A}$ ומשתמש בהתנהגות השונה שלו כדי לנחש עם איזו פונקציה הוא עובד. אם $\mathcal{A}$ מזייף הרבה יותר טוב עם F_k מאשר עם f, אזי D יאמר: "אם $\mathcal{A}$ הצליח לזייף, כנראה האורקל הוא F_k".

עכשיו ההסבר שממשיך כאן עם אנלוגיה הוא כי ממש קשה להבין את סגנון ההוכחה הזה, אתם כמובן יכולים לדלג הלאה. נבנה את זה עם סיפור - ונבין: יש לנו זוג עולמות, שנבדלים בדבר אחד בלבד - הפונקציה שהם משתמשים בה. בעולם האמיתי משתמשים בF_k ובאידיאלי בf. אנחנו מאמינים כי לא ניתן להבדיל בין F_k לf. עכשיו - נניח שמישהו בא וטוען: לא נכון, היריב $\mathcal{A}$ כן מרגיש הבדל ביניהם. הוא מזייף הרבה יותר טוב למשל בעולם האמיתי מהאידיאלי. אני רוצה לטעון - הוא טועה, לא הגיוני. כיצד? אנחנו נוכיח, שאם הוא צודק נובע מזה דבר שאנחנו יודעים שבלתי אפשרי. זה יפיל כמובן את הטענה שלו. מהו הדבר הבלתי אפשרי? **אף** אחד לא יכול להסתכל על הפונקציה ולדעת אם היא F_k או f. זה הדבר האסור. אז המטרה שלנו - לקחת את $\mathcal{A}$, שלפי הטענה "מרגיש הבדל בין העולמות", ולהשתמש בו על מנת לבנות מישהו מבחין D שיודע להבדיל בין העולמות. אם נצליח לבנות מבחין כזה D, סתרנו את ההבטחה. ואז: סתירה - אין הבדל בין העולמות. וקיבלנו מה שרצינו. כיצד בונים את המזהה הזה? נבנה דמות חדשה ושמה D. נותנים לה פונקציה מסתורית - אורקל - $\mathcal{O}$. אומרים לו: זה או F_k, או f: תגיד לי מי מבניהם. D עצמו לא יודע כיצד לזהות, אבל יש לו את $\mathcal{A}$: שלפי הטענה מההנחה בשלילה כן יש לו יכולת להבדיל. אז $\mathcal{D}$ מרמה את $\mathcal{A}$: הוא מריץ את $\mathcal{A}$ ומשחק מולו את משחק הזיוף - בכל פעם ש$\mathcal{A}$ צריך תג, D נותן לו אותו דרך הפונקציה המסתורית. בעצם נעצור להבין: D מריץ את היריב, כי היריב לכאורה הוא זה שיודע להבדיל בין הפונקציות. אבל - נחשוב מה $\mathcal{A}$ חווה: אם $\mathcal{O}$ היא באמת F_k - כל התגים הם $F_k(m)$. זהו העולם האמיתי. $\mathcal{A}$ לא יודע שהוא מדבר עם D, הוא חושב שהוא במשחק רגיל בעולם האמיתי כי הוא בסמלוץ. אם $\mathcal{O}$ היא f כל התגים הם $f(m)$ וזהו העולם האידיאלי. כלומר, D שתל את הפונקציה המסתורית בתוך המשחק של $\mathcal{A}$ ולכן העולם ש$\mathcal{A}$ רואה נקבע לפי מה ש$\mathcal{O}$ באמת. ואיך D מנצל את זה? הוא מתבונן: האם $\mathcal{A}$ הצליח לזייף? לפי הטענה של ההוא מהויכוח - $\mathcal{A}$ מזייף הרבה בעולם האמיתי ומעט בעולם האידיאלי כי יש הבדל ביניהם (בלי הגבלת הכלליות - יש פער ביניהם, אז נניח והוא ככה. בהוכחה המקורית זה עם ערך מוחלט, ואז אין צורך בחלוקה למקרים). אזי: אם $\mathcal{A}$ הצליח לזייף, כנראה היה בעולם האמיתי ולכן D ינחש F_k. אם נכשל: כנראה היה באידיאלי ולכן D ינחש f. וזה עובד - אם הפער בין העולמות גדול: אזי הניחוש של D נכון לעיתים קרובות. כלומר D אכן מצליח לזהות מי הפונקציה. אבל זו הסתירה - זה הדבר הבלתי אפשרי: לכן המסקנה שהיריב מרגיש הבדל בין העולמות - שגויה. אין הבדל בין העולמות, ומקבלים מה שרוצים.
נחזור להוכחה הפורמלית. ישנם שני מקרים אם ככה,

- האורקל של המבחין הוא פונקציה פסאודו־אקראית: נשים לב כי $\mathcal{O}(\cdot) = F_k(\cdot)$ גורר כי המבחין מסמלץ ל$\mathcal{A}$ בדיוק את המשחק $\text{Mac-forge}_{\mathcal{A},\Pi}(n)$ שכן גישת האורקל מעבירה את t ב$F_k(m)$. לכן:

$$\Pr_{k\in\{0,1\}^n}\left[D^{F_k(\cdot)}(1^n)=1\right] = \Pr\left[\text{Mac-forge}_{\mathcal{A},\Pi}(n)=1\right]$$

נעיר כי D המבחין חייב לקבל את n - פרמטר הבטיחות, באופן אונארי.

- מקרה שני הוא שהאורקל של המבחין הוא פונקציה אקראית: במקרה זה $\mathcal{O}(\cdot) = f(\cdot)$ עבור פונקציה $f \in \text{Func}_n$. המבחין במקרה זה מסמלץ ל$\mathcal{A}$ בדיוק את $\text{Mac-forge}_{\mathcal{A},\tilde{\Pi}}(n)$ שכן שם הוא מחזיר $f(m)$ פשוט. ולכן:

$$\Pr_{k\in\{0,1\}^n}\left[D^{f(\cdot)}(1^n)=1\right] = \Pr\left[\text{Mac-forge}_{\mathcal{A},\tilde{\Pi}}(n)=1\right]$$

מכאן, כיוון שF היא פונקציה פסאודו אקראית, אנו יודעים כי ההפרש בין המקרים זניח. כלומר קיימת פונקציה זניחה negl כך ש:

$$|\Pr_{k\in\{0,1\}^n}\left[D^{F_k(\cdot)}(1^n)=1\right] - \Pr_{k\in\{0,1\}^n}\left[D^{f(\cdot)}(1^n)=1\right]| \leq \text{negl}(n)$$

במקרה שלנו:

$$|\Pr\left[\text{Mac-forge}_{\mathcal{A},\Pi}(n)=1\right] - \Pr\left[\text{Mac-forge}_{\mathcal{A},\tilde{\Pi}}(n)=1\right]| \leq \text{negl}(n)$$

כמו שרצינו להוכיח. לכן: מתקיים כי

$$\Pr\left[\text{Mac-forge}_{\mathcal{A},\Pi}(n)=1\right] \leq \text{negl}(n) + \Pr\left[\text{Mac-forge}_{\mathcal{A},\tilde{\Pi}}(n)=1\right]$$

והרי הוכחנו $\Pr\left[\text{Mac-forge}_{\mathcal{A},\tilde{\Pi}}(n)=1\right] = 2^{-n}$, לכן:

$$\Pr\left[\text{Mac-forge}_{\mathcal{A},\Pi}(n)=1\right] \leq \text{negl}(n) + 2^{-n} = \text{negl'}(n)$$

שהיא זניחה גם כן. ולכן, קיבלנו את הדרוש. כלומר, לפי הגדרה הבניה היא Mac בטוחה.

0.2.2 הרחבת דומיין לMac

עד כה דיברנו על Mac לבלוק אחד, באורך n. אך מה אם רוצים לאמת הודעות ארוכות שמחולקות לכמה בלוקים? נבנה זאת בהדרגה.

יהי Mac' הMac הבסיסי לבלוק בודד, זה שהוכחנו כאן מעלה. נרצה לשאול כיצד להשתמש בו על מנת לאמת הודעה $m_1, ..., m_\ell$ של ℓ בלוקים.

נסיון ראשון. נצמיד תג נפרד לכל בלוק $t_i = \text{Mac}'_k(m_i)$. שולחים את ההודעה עם כל התגים.

החור: ערבוב בלוקים, כל תג מחובר לבלוק שלו אך לא למיקום שלו. יריב שרואה הודעה תקינה, יכול לשנות את סדר הבלוקים של ההודעה ולשמור על כל תג צמוד לבלוק שלו ־ הכל עדיין מאומת. הוא יכל להפוך משפט כמו "העבר 100 שקל מחשבון A לחשבון B" ל"העבר 100 שקל מחשבון B לחשבון A" ללא זיוף. בקיצור ־ לא עובד.

נסיון שני. מוסיפים אינדקס, נצמיד לכל בלוק גם את המיקום שלו לפני הMac. כלומר:

$$t_i = \text{Mac}'_k(i||m_i)$$

כעת הבלוק הi יודע כי הוא הבלוק ה$-i$. מה החור? חיתוך. פתרנו ערבוב, אך כעת היריב יכול לזרוק בלוקים מהסוף ־ בהינתן תגים ל$\{m_i\}_{i=1}^{\ell}$ הוא לוקח רק את החלק הראשון עם התגים שלו $m_1, ..., m_i$ וזו הודעה קצרה יותר עם תגים תקינים. למשל: "העבר מאה שקל, אבל אל תבצע אם היתרה שלילית" יכול להפוך ל"העבר מאה שקל".

נסיון שלישי. מוסיפים גם את אורך ההודעה. מה כאן? נצמיד לכל בלוק את אורך ההודעה.

$$t_i = \text{Mac}'_k(\ell||i||m_i)$$

כעת כל בלוק יודע מה מספר הבלוקים, אם יריב יזרוק בלוקים הבלוקים שישארו עדיין יטענו ℓ מקורי והמקבל יראה שחסרים בלוקים ־ זה חוסם את החיתוך. אך מה החור? הרכבה בין זוג הודעות. עדיין ניתן לערבב בלוקים בין שתי הודעות שונות באותו האורך. נניח כי היריב ראה תגים לשתי הודעות $\{m_i\}_{i=1}^{\ell}$ ו$\{m'_i\}_{i=1}^{\ell}$ שתיהן באורך ℓ. הוא יכול לבנות הודעה חדשה שמשלבת את שתיהן, למשל $m'_1, m_2, m'_3, m_4, ...$ ־ הכל מאומת. הבעיה היסודית כאן: שום דבר לא קושר את הבלוקים של הודעה אחת זה לזה, כל בלוק מאומת בנפרד. אז אפשר לקחת בלוקים שונים מהודעות שונות ולהרכיב.

מה הפתרון? מזהה אקראי משותף. נותנים לכל הודעה מספר אקראי משלה r. ומכניסים אותו לכל הבלוקים.

$$t_i = \text{Mac}'_k(r||\ell||i||m_i)$$

שולחים $(r, t_1, ..., t_\ell)$ והמקבל בודק שכל בלוק מאומת עם אותו r. למה זה סוגר את החור האחרון? כי r מקשר את כל הבלוקים של הודעה אחת באמצעות אקראיות. לכל הודעה יש מזהה יחודי משלה, לכן הבלוקים של הודעה אחת יחזיקו באותו מזהה. מה המחיר? התג ארוך בהרבה מההודעה, בהכרח. כל בלוק צריך תג משלו, וגם את השדות הנוספים. כלומר ־ אפשר ככה, זה לא יעיל. בעיקר רצינו ללמוד על המגבלות כאן וכיצד פותרים אותם. דרך יעילה יותר היא זו שנדבר עליה עכשיו:

0.2.3 CBC MAC

עוד לפני שנראה את הסכימה, נציין כי הסכימה בטוחה רק עבור הודעות מאורך קבוע: שהוא לא בלוק יחיד, אבל מס' הבלוקים בו תמיד זהה. כלומר יש לו ℓ כלשהו בלוקים וגודל בלוק n. מסתכלים על מרחב הודעות $\mathcal{M} = \{0,1\}^{\ell \cdot n}$.

תהי F פונקציה פסאודו אקראית, ויהי ℓ.
- $\text{Gen}(1^n)$: נגריל מפתח $k \in \{0,1\}^n$.
- $\text{Mac}_k(m)$: עבור הודעה $m \in \{0,1\}^{\ell \cdot n}$ ומפתח $k \in \{0,1\}^n$ נכתוב $m = (m_1, .., m_\ell)$ כאשר כל $i \in [\ell]$ מקיים כי m_i בגודל n. נגדיר $t_0 = 0^n$ אזי לכל $i \in [\ell]$ נגדיר כי $t_i = F_k(t_{i-1} \oplus m_i)$.
- $\text{Vrfy}_k(m,t)$: אם m הוא לא מאורך $\ell \cdot n$ נחזיר 0. אחרת, נחזיר 1 אם ורק אם $t = \text{Mac}_k(m)$.

משפט 7. יהי $\ell = \ell(n)$ פולינום. אם F היא פונקציה פסאודו־אקראית אזי הבניה לעיל היא בניה של סכימה לאימות הודעות, עבור מרחב ההודעות $\mathcal{M} = \{0,1\}^{\ell \cdot n}$.

מה קורה כאן? בכל שלב מבצעים XOR על הבלוק הנוכחי עם התוצאה הקודמת. התג הוא הבלוק האחרון בלבד. כלומר t_ℓ. אינטואיציה מדוע זה בטוח: בגלל השרשור, כל בלוק משפיע על כל מה שיבוא אחריו והתג הסופי תלוי בכל הבלוקים ובסדר שלהם. אי אפשר לערבב, לחתוך או להרכיב כי כל שינוי בבלוק כלשהו משנה את החישוב מאותה נקודה והלאה. זה חוסם את כל ההתקפות שנידונו לעיל.

שאלה שנוכל לשאול את עצמנו היא מדוע זה מוגדר רק עבור הודעות באורך קבוע - זו שאלה למחשבה. **ואולי למבחן?**

0.2.4 תקשורת פרטית ומאומתת - לשים לב אם בהרצאה הוא מפרמל יותר!

הנושא האחרון שנדון בו במסגרת הרצאה זו הוא Authenticated Encryption. כיצד משלבים הצפנה - סודיות, עם Mac - אימות על מנת לקבל ערוץ פרטי שהוא גם מאובטח וגם מאומת.

בידינו שני כלים מוכנים - Π_E הצפנה בטוחה CPA אשר נותנת סודיות (שאלה: למה דווקא CPA? כי היא גוררת בטיחות להודעות מרובות וכן נזכר כי CCA היא מה שאנו בונים כאן בשלבים אז לא יכולנו להתחיל ממנה, וΠ_{Mac} שהוא Mac בטוח שנותן אימות. אנו מעוניינים לשלב אותם, ובכן ישנן 3 דרכים טבעיות לעשות זאת ורק אחת מהן באמת בטוחה.

דרך ראשונה. Encrypt-and-MAC: בדרך זו מצפינים ומאמתים בנפרד. כלומר, שולחים את ההודעה ואת התג של ההודעה כל אחד בנפרד:

$$c \leftarrow \text{Enc}_{k_E}(m), t \leftarrow \text{Mac}_{k_M}(m)$$

שולחים (c,t).

מה הבעיה? הMac מחושב על ההודעה m הגלויה, והוא נשלח כמו שהוא. אבל Mac מבטיח אימות ולא סודיות, אין שום הבטחה כי t לא מדליף מידע על m. אפילו גרוע מכך - אם הMac דטרמיניסטי וזה ברוב המקרים כמו שראינו $t = F_{k_M}(m)$, אזי אותה הודעה תמיד נותנת את אותו התג. יריב שרואה שני תגים זהים יודע ששתי ההודעות זהות, זה בדיוק שובר CPA: ההצפנה הפכה לדטרמיניסטית ברכיב הt.

דרך שנייה. MAC-then-Encrypt: מה הרעיון כאן? מחשבים תג על m ואז מצפינים את ההודעה והתג יחד.

$$t \leftarrow \text{Mac}_{k_M}(m), c \leftarrow \text{Enc}_{k_E}(m||t)$$

שולחים את c, המקבל מפענח ומקבל את $m||t$ ומוודא שהתג מתאים.

עכשיו זה נראה יותר טוב - התג מוצפן, לכן אין דרך באמצעותו לגלות את ההודעה m. אך כעת המקבל צריך לפענח ורק אז לבדוק את התג - אם בפענוח יש כמה סוגי שגיאות: שגיאות ריפוד, שגיאות אימות וכו' והיריב יכול להבחין בניהן, חוזרים בדיוק להתקפת Padding Oracle שכבר ראינו - התוקף שולח צפנים מעוותים ולומד מהתגובות על ההודעה. זה קרה בפועל במציאות גם בכמה מקרים. נרחיב: בדרך זו המקבל קיבל צופן c, כדי לבדוק את התג הוא חייב קודם לפענח את הצופן c כי התג מוצפן בתוכו. בפענוח CBC ישנו שלב של הורדת ריפוד - והוא קורה לפני בדיקת התג: אז יריב ששולח צופן מעוות גורם לשרת לפענח ולבדוק ריפוד, ומגיב עם שגיאת ריפוד או שגיאת אימות. ההבדל בין זוג השגיאות הוא מה שמזין את הPadding oracle. השרת "עובד" על הצופן של היריב עוד לפני שהתג נבדק. בדרך השלישית שמיד נראה, התג מחושב על הצופן הגלוי אז המקבל יכול לבדוק אותו מבלי לפענח כלום. יריב ששולח צופן מעוות יתפס מיד בבדיקת התג והשרת דוחה אותו עוד לפני שיגע בריפוד. בדרך 2 הפענוח קורה לפני הבדיקה, ובדרך 3 הבדיקה קורית לפני הפענוח.

דרך שלישית. Encrypt-then-MAC: כאן אנחנו קודם מצפינים, ואז מאמתים את הצופן.
בהינתן הודעה m:

1. מצפינים את ההודעה $c \leftarrow \mathrm{Enc}_{k_E}(m)$
2. מחשבים תג על הצופן - לא על ההודעה: $t \leftarrow \mathrm{Mac}_{k_M}(c)$
3. שולחים את הזוג (c,t)

הנקודה המרכזית שמבדילה מדרכים קודמות - הMac מחושב על הצופן c, הצופן הגלוי. ולא על m.
מה המקבל עושה? בודק את התג - מריץ $\mathrm{Vrfy}_{k_M}(c,t)$. אם התג לא תקין - הוא ידחה מיד. אחרת, הוא תקין: אז הוא מפענח $m = \mathrm{Dec}_{k_E}(c)$.

כלומר: הסוד של בדיקה לפני פענוח - הוא הקסם.

מדוע זה בטוח? מה ששבר את כל הסכימות הקודמות בCCA הוא שליריב היה אורקל פענוח: הוא היה לוקח את צופן האתגר, משנה אותו קצת ליצור צופן חדש c' ושואל את האורקל מה הפענוח של c', ומזה מחלץ את ההודעה. כעת נסביר מדוע בדרך זו - האורקל של היריב הופך לחסר תועלת:

יריב רוצה לשאול את אורקל הפענוח על צופן כלשהו. אך בדרך זו, אורקל הפענוח קודם בודק את התג: אז בשביל שהשאלה תחזיר משהו מועיל , ולא סתם נדחה - היריב צריך לשלוח זוג (c',t') שבו התג תקין. כלומר t' הוא תג חוקי על c'. יש שתי אפשרויות למה שהיריב יכול לשלוח:

1. c' שהוא כבר קיבל מאורקל ההצפנה: אבל אז הוא כבר יודע מה ההודעה שלו אז הפענוח לא לימד אותו שום דבר מחדש.
2. c' חדש שהוא בנה או עיוות בעצמו: כדי שזה יעבור את בדיקת התג, הוא צריך תג t' תקין על c' החדש. אבל הMac בטוח - כלומר לא ניתן לזייף תג חדש על צופן חדש מבלי לדעת את k_M. לכן הוא לא יכול ליצור זוג כזה - כל נסיון כזה יתפס בבדיקת התג ויזרק.

המסקנה: היריב לא יכול לקבל מהאורקל שום פענוח מועיל. אורקל הפענוח, שהיה הכלי ששבר את כל הסכימות הקודמות מפסיק לעזור.

לכן, השילוב הנ"ל לא נותן רק אימות אלא **סכמה בעלת בטיחות CCA מלאה.**

אבל - הMac צריך להיות Strong Mac, ובפרט דטרמיניסטי. כלומר - אי אפשר ליצור תג חדש אפילו לצופן שכבר יש לו תג.

מדוע אנחנו צריכים שני מפתחות שונים בלתי תלויים?

אנחנו השתמשנו במפתח אחד להצפנה, ואחד לאימות: k_M, k_E . בהרבה מהמקרים, המפתחות נראים אותו דבר - איבר אקראי ב$\{0,1\}^n$. במקרה כזה, טבעי להשתמש באותו מפתח לשניהם. אבל בפועל פרקטית זה גרוע, ותמיד מומלץ להפריד בין זוג מפתחות של סכימות שונות.

למשל, נניח פונקציה פרמוטציה פסאודו־אקראית, אזי ניתן לבנות סכימת הצפנה כזו:

$$\mathrm{Enc}_k(m;r) = F_k(m||r)$$

היא בטוחה. וניתן גם לבנות סכימת אימות כזו:

$$\mathrm{Mac}_k(m) = \mathrm{F}_k^{-1}(m)$$

נשים לב כי לפי השיטה השלישית Encrypt-then-MAC:

$$c = \mathrm{Enc}_k(m;r) = F_k(m||r)$$

$$t = \mathrm{Mac}_k(c) = \mathrm{F}_k^{-1}(F_k(m||r)) = m||r$$

ואז התג יצא $m||r$ - כלומר ההודעה עצמה בגלוי. הפונקציות F_k וF_k^{-1} ביטלו זו את זו - כי מדובר באותו מפתח k(!)
לכן הלקח הוא: גם אם שני הרכיבים בטוחים - עדיף להשתמש במפתחות שונים.

0.3 תרגילים

נראה מס' שאלות בסיסיות יותר או פחות על חומר ההרצאה.

לפני כן, תבנית כללית על שאלות מסוג האם Mac בטוח?

אם התשובה היא לא - רוצים למצוא יריב אחד לפחות שמזייף בהצלחה. זיוף מוצלח: זוג (m,t) כך ש:

1. t הוא תג תקין על m. כלומר Vrfy מחזיר 1.
2. m היא הודעה חדשה, $m \notin Q$. לא נשאל על ידי האורקל עליה.

מה היריב מחזיר? תמיד זוג (m,t) באשר m היא ההודעה שהוא מזייף וt הוא התג המזויף.

היריב יכול לעשות בכל דרך: לשאול את האורקל שאלות, ולקבל תגים תקינים. המטרה היא לנצל את התגים הללו כדי לבנות תג להודעה החדשה.

בעיה 8. נגדיר את הסכימה הבאה: יהי $m \in \{0,1\}^{k \cdot n}$ ונגדיר $\mathrm{Mac}_k(m) = m_1 \oplus ... \oplus m_k \oplus k$ - כלומר חלוקת ההודעה לk מקטעים באורך n והוספת Xor ביניהם עם k.
באשר $|m_i| = n$ לכל $i \in [k]$. האם הסכימה היא סכימה Mac העמידה בפני זיוף הודעות?
פתרון: לא. יריב $\mathcal{A}$ יכול לעבוד ככה -
1. בחר וידוא עבור הודעה 0^{nk} מהאורקל. קבל חזרה: $\mathrm{Mac}_k(0^n) = 0^n \oplus ... \oplus 0^n \oplus k = k$. מה קרה כאן? האורקל החזיר את k בגלוי! זו החולשה.
2. בחר הודעה $m \in \{0,1\}^{nk}$ כרצונך כך ש$m \neq 0^{nk}$. חשב בעצמך את $t = \mathrm{Mac}_k(m) = m_1 \oplus ... \oplus m_k \oplus k$. כלומר, כעת שהיריב יודע את k הוא בוחר הודעה כלשהי ומחשב בעצמו את התג t - מדוע הוא יכול לעשות זאת? כי הוא יודע את k ויודע את $(m_1, ..., m_k)$.
3. החזר (m, t).
מדוע זה זיוף מוצלח? לזיוף מוצלח ישנם 2 תנאים:
- התג תקין: ואכן, היריב חישב אותו בדיוק לפי הנוסחה. אז Vrfy יעבור.
- ההודעה החדשה: $m \neq 0^{nk}$. הדבר היחיד שהיריב שאל עליו הוא 0^{nk}. לכן $Q = \{0^{nk}\}$. לכן $m \notin Q$.
שני התנאים מתקיימים, אזי היריב הצליח בהסתברות 1. לכן הסכימה לא בטוחה.
כלומר: מצאנו הודעה אחת שהוא לא שאל את האורקל עליה ישירות, והוא גילה עליה את הטאג t. לכן הסכמה לא בטוחה.

בעיה 9. תהי F פונקציה פסאודו־אקראית. נרצה לאמת הודעה $m_0||...||m_\ell$ כאשר $m_i \in \{0,1\}^n$ לכל $i \in [\ell]$. נחשב $t = F_k(m_1) \oplus .... \oplus F_k(m_\ell)$. האם הסכימה בטוחה?
פתרון: לא, כיוון שאין כל חשיבות לסדר. כלומר $a \oplus b = b \oplus a$ ולכן הסדר הוא לא משנה.
היריב בוחר הודעה כלשהי $m = m_0||....||m_\ell$. הוא שואל עליה את האורקל, ומקבל תג תקין:

$$t = F_k(m_1) \oplus F_k(m_2) \oplus .... \oplus F_k(m_\ell)$$

כעת נבנה זיוף. נשנה את הסדר: היריב יחזיר את ההודעה עם סדר שונה ואת t. למשל:

$$(m_2||m_1||m_3||.....||m_\ell, t)$$

כעת נוכיח שזה זיוף תקין. ראשית, הודעה זו שונה - והיא לא נשאלה ע"י האורקל. שנית - הטאג של ההודעה הוא:

$$t' = F_k(m_2) \oplus F_k(m_1) \oplus .... \oplus F_k(m_\ell) = F_k(m_1) \oplus F_k(m_2) \oplus .... \oplus F_k(m_\ell) = t$$

ולכן אכן החזרנו זוג עם הודעה וטאג שמתאים להודעה. היריב מצליח בהסתברות 1, והסכימה לא בטוחה.

בעיה 10. תהי F פונקציה פסאודו־אקראית. האם הMac הבא בטוח? נניח $m = m_1||m_2$ כך ש $m_1, m_2 \in \{0,1\}^n$.

$$t = (F_k(m_1), F_k(F_k(m_2)))$$

פתרון: לא, הסכימה לא בטוחה. נתבונן ביריב:
- $\mathcal{A}$ יקרא שתי הודעות m_1, m_2 וישאל את האורקל על m_1m_2 ויקבל חזרה $t = (F_k(m_1), F_k(F_k(m_2)))$.
- $\mathcal{A}$ יקרא שתי הודעות m_2, m_3 וישאל את האורקל על m_2m_3 ויקבל חזרה $t = (F_k(m_2), F_k(F_k(m_3)))$
- $\mathcal{A}$ יחזיר $m' = m_1||m_3$ עם התג $(F_k(m_1), F_k(F_k(m_3)))$

נבחין כי:
1. $m' \notin Q$ שכן לא שאל את האורקל על m_1m_3. 2. וכן, נבחין כי יש לנו את $(F_k(m_1), F_k(F_k(m_3)))$ מהשאלות הקודמות שלוץ

לכן, מצאנו יריב שמצליח בהסתברות 1 למצוא זוג הודעה וטאג, ולא בטוח Mac.

בעיה 11. תהי F פונקציה פסאודו־אקראית. האם הMac הבא בטוח? נניח $m = m_0||m_1$ כך ש $m_1, m_0 \in \{0,1\}^{n-1}$.

$$t = (0||m_0||F_k(1||m_1))$$

פתרון: לא, הסכימה לא בטוחה. נתבונן ביריב:
- $\mathcal{A}$ יקרא שתי הודעות m_0, m_1 וישאל את האורקל על m_0m_1 ויקבל חזרה $t = (0||m_0||F_k(1||m_1))$
- $\mathcal{A}$ יקרא שתי הודעות m_1, m_2 וישאל את האורקל על m_2m_3 ויקבל חזרה $t' = (0||m_1||F_k(1||m_2))$
- $\mathcal{A}$ יחזיר $m' = m_0||m_2$ עם התג $(0||m_0||F_k(1||m_2))$

נבחין כי:
1. $m' \notin Q$ שכן לא שאל את האורקל על m_0m_2. 2. וכן הצלחנו למצוא את ההודעה כי אנחנו יודעים שניתן להתחיל בקריאה של n תווים ראשונים בt ואז להפסיק. זה $m_0||0$. ואז - אנחנו יכולים לקרוא את החצי האחרון של הביטים בt' ולשרשר.

לכן, מצאנו יריב שמצליח בהסתברות 1 למצוא זוג הודעה וטאג, ולא בטוח Mac.

בעיה 12. ראינו שמטודולוגית encrypt-and-authenticate לא תמיד מובילה לסכימה בטוחה. אבל, אין זה אומר שהיא לא עובדת לפעמים עבור סכימות מסוימות. נתבונן בסכימה הבאה כאשר F היא פונקציה פסאדו־אקראית: נגדיר $\text{Enc}_{K_1}(m) = \langle r, F_{K_1}(r) \oplus m \rangle$ ו־$\text{Mac}_{K_2}(m) = F_{K_2}(m)$.

1. האם סכימת ההצפנה שמתקבלת על ידי הפעלת מטודולוגית encrypt-and-authenticate היא בטוחה CPA?
2. האם הסכימה שמתקבלת על ידי הפעלת מטודולוגית encrypt-and-authenticate היא סכימת אימות הודעות בטוחה?

פתרון לסעיף 1: הסכימה אינה בטוחה CPA. נשים לב שאלגוריתם ההצפנה הינו:

$$\text{Enc}_{K_1,K_2}(m) = \langle \text{Enc}_{K_1}(m), \text{Mac}_{K_2}(m) \rangle$$

ושסכימת ה־Mac היא דטרמיניסטית. נראה יריב שמצליח במבחן CPA:

- $\mathcal{A}$ יבחר באופן שרירותי שתי הודעות m_0, m_1 מאותו האורך.
- $\mathcal{A}$ יבקש מאורקל ההצפנה שלו הצפנה עבור שתי ההודעות m_0, m_1 ויקבל חזרה $c_0 = (c_0^{enc}, c_0^{mac})$ ו־$c_1 = (c_1^{enc}, c_1^{mac})$.
- $\mathcal{A}$ יחזיר לניסוי את זוג ההודעות m_0, m_1 ויחכה לקבלת צופן האתגר $c = (c^{enc}, c^{mac})$.
- אם $c^{mac} = c_0^{mac}$, $\mathcal{A}$ יחזיר 0. אחרת, יחזיר 1.

קל לראות שהיריב יצליח בהסתברות 1.

פתרון לסעיף 2: הסכימה היא סכימת Mac בטוחה.

הוכחה: למעשה, יש לנו את סכימת ה־Mac הבאה: $\text{Mac}_k(m) = ((r, F_{k_1}(r) \oplus m), F_{k_2}(m))$. נסמן את הסכימה הזו ב־$\Pi$. נראה שאם הסכימה הזו אינה בטוחה, אזי גם סכימת Π' המוגדרת על ידי $\text{Mac}'_{K_2}(m) = F_{K_2}(m)$ אינה בטוחה, בסתירה להנחה שהיא בטוחה.

לפי הנחת השלילה Π אינה בטוחה, ולכן קיים אלגוריתם $\mathcal{A}$, פולינום $p(\cdot)$, ועבור אינסוף n־ים מתקיים:

$$\Pr\left[\text{Mac-forge}_{\mathcal{A},\Pi}(n)=1\right]\geq\frac{1}{p(n)}$$

נראה יריב $\mathcal{A}'$ לסכימה Π' בניסוי $\text{Mac-forge}_{\mathcal{A}',\Pi'}(n)$:

- **קלט:** פרמטר בטיחות 1^n.
- **גישת אורקל:** היריב מקבל גישת אורקל ל־$\text{Mac}'_K(m)=F_{K_2}(m)$.
- $\mathcal{A}'$ מגריל מפתח חדש $k_1\leftarrow\{0,1\}^n$. הוא מריץ את $\mathcal{A}$ על קלט 1^n ועונה על שאילתות האורקל שלו, $\text{Mac}_{k_1,k_2}(\cdot)$. כאשר $\mathcal{A}$ שואל על הודעה m, היריב $\mathcal{A}'$ ישאל את האורקל שלו על m ויקבל חזרה $F_{K_2}(m)$. הוא יגריל באקראי $r\leftarrow\{0,1\}^n$ ויחזיר ל־$\mathcal{A}$ את הערך:

$$\langle r,F_{K_1}(r)\oplus m,F_{K_2}(m)\rangle$$

- כלומר, $\mathcal{A}'$ יודע לענות על השאילתות ש־$\mathcal{A}$ מוציא, שכן הוא יודע לייצר:

$$\text{Mac}_k(m)=\langle r,F_{k_1}(r)\oplus m,F_{k_2}(m)\rangle$$

- כאשר $\mathcal{A}$ יוציא זוג $\langle m,t\rangle$, היריב $\mathcal{A}'$ יוודא שההודעה לא נשאלה קודם. לאחר מכן נסמן $t=\langle t_1,t_2,t_3\rangle=\langle r,m\oplus F_{K_1}(r),F_{K_2}(m)\rangle$. אם מתקיים $F_{K_1}(t_1)\oplus m=t_2$, היריב יוציא את הזוג $\langle m,t_3\rangle$. אחרת, יוציא $\perp$.

ניתוח הצלחה: נשים לב ש־$\mathcal{A}'$ מצליח בניסוי אם ורק אם $\mathcal{A}$ מצליח בניסוי. לכן:

$$\Pr\left[\text{Mac-forge}_{\mathcal{A},\Pi}(n)=1\right]=\Pr\left[\text{Mac-forge}_{\mathcal{A}',\Pi'}(n)=1\right]\geq\frac{1}{p(n)}$$

עבור אינסוף n־ים, בסתירה להנחה ש־Π' בטוחה. כנדרש.
0>0